# Feature Ablation Study for Deep Learning Statistical Arbitrage on IDX Equities

trading-BEI project

July 2026

## Abstract

We describe the experimental setup for determining the optimal number and composition of input features for a deep learning statistical arbitrage (DLSA) strategy on Indonesia Stock Exchange (IDX) equities. Unlike the original DLSA pipeline, no factor model is used: normalized raw features are fed directly into a two-stage cross-sectional Transformer — temporal self-attention over each stock's 60-day window, then self-attention *across* all stocks on a given day — that learns representation and trading signal end-to-end. The model is trained with a DLSA-style economic objective: maximize the annualized Sharpe ratio of a long-only softmax portfolio, *net* of transaction costs, over blocks of consecutive trading days so that turnover is genuinely penalized. Because the deployed strategy is *long-only*, it carries market beta and is therefore benchmarked against a buy-and-hold IHSG index. We use a nested, group-wise ablation across five economically motivated feature groups, run over eight random seeds per configuration, and evaluated on strictly time-ordered walk-forward out-of-sample data. The optimal feature set is identified as the point at which the walk-forward Sharpe ratio saturates before additional features degrade performance through overfitting.

## 1 Introduction

The `trading-BEI` project adapts Deep Learning Statistical Arbitrage (Guijarro-Ordonez, Pelger & Zanotti, 2021) to IDX equities. The key deviation from upstream DLSA is the removal of the factor model and residual stages: raw daily stock-summary data is normalized and passed directly to a Transformer trading policy.

A central open question is how many input features the model should receive. Because there is no factor model to pre-extract structure, the Transformer must learn cross-asset co-movement itself. Adding features increases model capacity but, on a limited history of roughly 1,250 trading days, also increases the risk of overfitting. This report specifies a controlled ablation study to locate the optimal feature count and composition.

## 2 Objective

The project has two nested objectives. The *research* objective is to identify how many and which input features maximize risk-adjusted out-of-sample performance under realistic IDX trading frictions — the ablation reported here. The *model* objective, optimized during training, is economic rather than statistical: rather than regressing next-day returns, the network directly maximizes the annualized Sharpe ratio of the long-only portfolio it induces, net of transaction costs. Formally, for a block of consecutive rebalance periods the training loss is $-$Sharpe of the net excess-return series (Section 7). This aligns the training signal with what is actually traded and evaluated, and — because costs are charged inside the block — makes the model pay for churn. The headline decision criterion for the ablation is the walk-forward net Sharpe of the traded top-$N$ rule; cumulative return versus a buy-and-hold IHSG benchmark, maximum drawdown, and turnover are reported alongside it.

## 3 Data

The raw input is the IDX daily *Ringkasan Saham* (stock summary), one record per (date, ticker), spanning roughly six years (2020-07 to 2026-07). After cleaning, the panel retains OHLC prices, previous close, volume, value, trade frequency, foreign buy/sell, listed and tradeable shares, and end-of-day bid/offer with sizes. A single processed `panel.parquet` feeds all experiments.

**Universe.** A causal liquidity screen is applied: a name is tradable on day $t$ only if its trailing 20-day *median* daily traded value is at least 1 billion IDR (`market.apply_universe`). This keeps penny stocks out of both the tradable set and the per-day z-score statistics.

**Corporate actions.** IDX's raw `close` is not split-adjusted, but the reported `Previous` (prev_close) is adjusted on the ex-date, so daily returns use $\ln(C_t/\text{prev}_t)$ (falling back to the lagged close), turning a 1:10 split into a $\approx 0\%$ return rather than a spurious $-90\%$. An adjusted log-price path drives all multi-day quantities so momentum and the forward target are split-consistent.

**Data-quality guards.** The raw `OpenPrice` is frequently zero; open-based and range features mask non-positive prices to `NaN` (later neutralized to 0 after standardization) rather than dropping rows. Rows are dropped only when the core feature (`log_return`) is missing, so the sample set is invariant to which optional feature groups are active.

# 4 Strategy Setting

The strategy is *long-only*: positions lie in $[0, 1]$ rather than the dollar-neutral $[-1, 1]$ of classical statistical arbitrage. The model head therefore uses a sigmoid (or clamped `tanh`), and weights are normalized to a fully-invested (or cash-permitting) book.

A direct consequence is exposure to market beta: unlike a market-neutral long/short book, a long-only strategy moves with the index. The appropriate benchmark is therefore a buy-and-hold IHSG index, not an absolute Sharpe threshold. We recommend training with the full cross-sectional signal (allowing the model to learn both out- and under-performers) and applying the long-only constraint only at the portfolio-construction stage.

# 5 Model Architecture

The flagship model is a *cross-sectional* Transformer (`models/cross_sectional.py`) that processes one trading day at a time. Its input is the tensor of all $N$ eligible stocks on that day, each with a $T$=60-day lookback window of $F$ standardized features, shape $(N, T, F)$; the output is one score per stock, $(N, )$, which is ranked to select the long book. The forward pass has three stages:

1. **Input projection + temporal encoding.** A linear map $\mathbb{R}^F \to \mathbb{R}^d$ plus a learned positional embedding over the $T$ time steps. A stack of `temporal_layers` standard Transformer encoder layers applies self-attention *over the 60 days* of each stock independently, yielding a $(N, T, d)$ tensor.

2. **Pooling.** The last time step is taken (`pooling: last`), collapsing each stock to a single $d$-dimensional vector, $(N, d)$.

3. **Cross-sectional encoding + head.** The $N$ stock vectors are treated as an unordered set (no positional encoding) and passed through `cross_layers` Transformer encoder layers of self-attention *across stocks*, so each stock's representation is informed by the whole market that day. A `LayerNorm` + linear head emits one scalar score per stock.

The cross-sectional stage is the key departure from the per-stock baseline Transformer (`models/transformer.py`), which scores each stock in isolation; here scoring is a *relative* comparison across the day's cross-section, and there is no factor model — attention runs directly on normalized raw features. Table **??** lists the architecture hyperparameters, held fixed across all ablation experiments.

Table 1: Architecture and optimization settings (locked across A–E).

| Setting | Value |
|---|---|
| Model width $d_{\text{model}}$ | 64 |
| Attention heads | 4 |
| Temporal / cross layers | 2 / 2 |
| Feed-forward dim | 128 |
| Dropout | 0.1 |
| Lookback $T$ / horizon | 60 / 1 |
| Pooling | last step |
| Optimizer | Adam |
| Learning rate | $3\times10^{-4}$ |
| Weight decay | $1\times10^{-5}$ |
| Epochs (max) | 40 |
| Early-stop patience | 8 |
| Batch granularity | 1 day / step |

# 6 Training Objective

The model is trained with a DLSA-style economic objective (`train_dlsa`) rather than regression. Each gradient step draws a block of $K$=32 *consecutive* trading days. On each day the model scores every stock; a softmax over the cross-section (temperature $\tau$=0.1, concentrating weight like the traded top-$N$ book) produces long-only weights $w \in [0, 1]^N$ that sum to one. An extra always-zero "cash" logit (`allow_cash`) lets the portfolio retreat to the risk-free rate. Day-over-day weight changes within the block, aligned by ticker, are charged asymmetric buy/sell costs, and the loss is the negative annualized Sharpe of the block's *net* excess returns:

$$\mathcal{L} = -\frac{\overline{r}^{\,\text{net}} - r_f}{\text{std}(r^{\text{net}})}\sqrt{A}, \qquad r_t^{\text{net}} = w_t^\top \rho_t - c_t,$$

where $\rho_t$ are simple returns, $c_t$ the ticker-aligned turnover cost, and $A$ the number of rebalance periods per year. Model selection and early stopping use the net Sharpe of the *actually traded* rule — top-$N$ equal weight after costs — on the validation days, so the model selected is the one truly traded, not the softmax portfolio.

# 7 Feature Groups

Ablation is performed over *groups* of economically related features rather than individual features, to limit the number of experiments and reduce noise. Each feature is standardized cross-sectionally within each trading day, following the DLSA convention. Table 1 lists the five groups.

Table 2: Feature groups used in the ablation.

| Grp | Features | # |
|---|---|---|
| G1 | log_return, mom_5/10/20 | 4 |
| G2 | hl_range, roll_vol_20, range_ratio | 3 |
| G3 | log_volume, log_value, turnover, amihud | 4 |
| G4 | foreign_flow_ratio, foreign_roll_5 | 2 |
| G5 | bid_ask_spread, book_imbalance | 2 |

Table 3: Nested experiment design.

| Exp | Active groups | ≈# feat. |
|---|---|---|
| A (baseline) | G1 | 4 |
| B | G1+G2 | 7 |
| C | G1+G2+G3 | 11 |
| D | G1+G2+G3+G4 | 13 |
| E (full) | G1+G2+G3+G4+G5 | 15 |

Representative feature definitions, all causal (no look-ahead):

$$r_t = \ln(C_t/C_{t-1}) \tag{1}$$

$$\text{hl\_range}_t = (H_t - L_t)/C_t \tag{2}$$

$$\text{turnover}_t = V_t/\text{shares}_t \tag{3}$$

$$\text{amihud}_t = |r_t|/\text{value}_t \tag{4}$$

$$\text{ff\_ratio}_t = (FB_t - FS_t)/(V_t + 1) \tag{5}$$

where $C, H, L$ are close/high/low, $V$ volume, $FB/FS$ foreign buy/sell.

# 8 Ablation Design

Experiments are *nested*: starting from a minimal set and adding one group at a time (Table 2). The optimal feature count is the stage at which the walk-forward Sharpe stops improving.

If the Sharpe rises through A→B→C then plateaus or declines at D/E, the saturation point identifies the optimal feature set. An optional leave-one-group-out pass on the best configuration confirms each group's marginal contribution.

# 9 Experimental Controls

For the comparison to be valid, *only the active feature list* may differ between experiments. Every other knob is locked in a shared base config (`configs/ablation/_base.yaml`); experiments A–E inherit it and override only `experiment_name` and `feature_groups`. Held identical are: the architecture and optimization settings of Table **??**; the lookback $T$=60 and horizon; the walk-forward split dates; the long-only, top-$N$=10 portfolio construction; the transaction-cost and turnover model; and the seed grid. Without these controls, differences in Sharpe cannot be attributed to features.

**Walk-forward splits.** Evaluation uses a rolling retrain. The out-of-sample period begins 2024-07-01 and is covered by four folds of six months each; every fold trains up to a six-month validation window placed immediately before its test period, trades the next six months out-of-sample, then rolls forward. Model weights are re-initialized with an identical seed per fold so differences come from data, not initialization, and the four folds' test scores are stitched into one two-year out-of-sample series before backtesting.

**Execution timing.** Features on day $t$ use day-$t$ closing data, so orders informed by them cannot execute at that same close. The target and the simulator share an execution lag of one day: decide after the close of $t$, execute at the close of $t$+1, over strictly consecutive trading days on which the name actually trades. This closes the most common look-ahead leak; `execution_lag`=0 (same-close MOC) is reserved for signal-decay diagnostics only.

**Costs.** IDX round-trip frictions are modeled as 15 bps commission on the buy side and 25 bps on the sell side (commission plus sell tax), against a risk-free rate of 5.5% (Sharpe is computed in excess of it). The realistic simulator additionally charges each name's own closing half-spread (default 35 bps when the book is empty, capped at 200 bps); the training loss uses the effective commission-plus-typical-spread number so it is not double-counted.

# 10 Evaluation

Evaluation is out-of-sample (walk-forward), never training loss. The primary ranking metric is the annualized Sharpe ratio; supporting metrics are cumulative return versus buy-and-hold IHSG (because long-only carries beta), maximum drawdown, and turnover (a proxy for the high real trading costs on IDX).

For robustness, each configuration is run over **eight random seeds**, reporting mean ± standard deviation. Eight seeds provide a stable variance estimate and support significance testing between configurations; a Sharpe gap below roughly 0.1 is likely seed noise rather than a feature effect. The full study therefore comprises $5 \times 8 = 40$ runs.

# 11 Implementation Changes

The study requires: (i) making `src/preprocess.py` config-driven, replacing the fixed `FEATURE_COLUMNS` constant with group definitions and computing the feature superset while selecting active groups from config; (ii)

fixing the `open=0` bug for open-based features; (iii) one YAML per experiment in `configs/` differing only in `feature_groups`; (iv) extending `compare.py` to aggregate results into a single table (feature set → Sharpe → return → drawdown → turnover, mean ± std across seeds); and (v) saving metrics and equity-curve plots to `results/`.

## 12 Risks

The main risks are overfitting from limited data combined with many features (precisely the effect under test), feature collinearity (e.g. log_volume vs. log_value should not be read as new information), seed noise (hence multi-seed runs), and look-ahead leakage in momentum, rolling, and normalization steps (the most bug-prone area, to be covered by anti-look-ahead unit tests). The long-only benchmark must remain IHSG, not an absolute return figure.

## 13 Definition of Done

The study is complete when all five experiments (A–E) have run over eight seeds each (40 runs), the comparison table is populated with mean ± std for all metrics, the optimal feature set is identified with justification (the Sharpe saturation point), and findings are documented in `results/` and summarized in the project plan.